

CSCE 611 OPERATING SYSTEMS · MODULE 01 · LECTURE NOTES

What an Operating System Is For

Abstraction, arbitration, and the illusions in between.

MODULE OUTCOMES

State the two jobs of an operating system and give examples of each.

Explain the kernel/user boundary and what enforces it.

Trace a system call from the user instruction to the kernel and back.

Explain why hardware support is required for isolation.

Describe the illusions an OS maintains and where each leaks.

1 The two jobs

Strip away the detail and an operating system does two things.

Job	Means	Examples
Abstraction	Turn awkward hardware into something a program can be written against.	A spinning disk with sectors and seek times becomes a file you read(). Physical RAM becomes a private address space. One CPU becomes many processes. A network card becomes a socket.
Arbitration	Decide who gets what, and prevent programs from interfering with each other.	Scheduling the CPU. Allocating memory. Enforcing file permissions. Ensuring one process cannot read another's data or crash the machine.

THE TWO ARE INSEPARABLE

Abstraction without arbitration is a library — useful, and no protection. Arbitration without abstraction is a traffic policeman with nothing to direct.

The kernel's actual work is maintaining a set of **illusions**: each program is given a convincing impression that it owns the machine. That illusion is what makes programs writable at all — the alternative is that every application must cooperate explicitly with every other, which is how computing worked before the mid-1960s and why it did not scale.

Every illusion leaks. Knowing where each one leaks is a large part of what distinguishes a systems programmer.

What a process believes	What is actually true	Where it leaks
I have the CPU to myself.	It gets a few milliseconds at a time, interleaved with others.	Timing jitter; cold caches after a context switch (CSCE 614 Module 05); unpredictable latency.
My address space is contiguous and starts at a fixed address.	Physical pages are scattered wherever they were free.	TLB misses (CSCE 614 Module 07); page faults; NUMA placement.
My memory is private.	Caches, branch predictors, and TLBs are shared hardware.	Spectre and cache side channels (CSCE 614 Module 04).
My write() reached the disk.	It is in a kernel buffer and may not be written for seconds.	Data loss on power failure. Module 09.
A file is a contiguous array of bytes.	It is a set of blocks scattered across a device.	Alignment effects; the cost of fsync; fragmentation.

2 The kernel/user boundary

Isolation cannot be achieved in software alone. Any check the kernel writes in software is an instruction a malicious program could simply not execute. The guarantee must come from hardware.

2.1 Privilege modes

Every processor intended to run an operating system provides at least two privilege modes:

- **User mode.** Ordinary computation only. Instructions that touch devices, modify page tables, disable interrupts, or change privilege level are forbidden and **trap** if attempted.
- **Supervisor (kernel) mode.** Everything is permitted.

THE ONE-WAY DOOR

The crucial property is not merely that user code cannot execute privileged instructions. It is that user code cannot *choose where it enters the kernel*.

At boot, the kernel installs a **trap vector** — a fixed address the hardware jumps to on any trap. A system call instruction transfers control to that address and nowhere else.

So the kernel is entered only at code the kernel wrote, with the hardware having already switched privilege level. Without this, a program could jump into the middle of a kernel routine past its validation checks, and no amount of careful kernel programming would help.

2.2 A system call, step by step

```

// User code
write(fd, buf, n);

// 1. libc loads the call number and arguments into registers:
//     a7 = SYS_write, a0 = fd, a1 = buf, a2 = n
// 2. executes ECALL (RISC-V) / SYSCALL (x86-64) / SVC (ARM)

//     HARDWARE: raise privilege to supervisor mode; jump to the
//     trap vector the kernel installed at boot.

// 3. Kernel saves the user register set into the process's trapframe.
// 4. Dispatches on the call number to sys_write().
// 5. VALIDATES: is fd a valid open file for this process?
//     does [buf, buf+n) lie entirely within this
//     process's address space?
// 6. Performs the operation.
// 7. Places the return value, restores registers, executes SRET.

//     HARDWARE: drop to user mode; resume at the instruction
//     after the ECALL.

```

STEP 5 IS THE SECURITY BOUNDARY

Every argument to a system call is **attacker-controlled data**. A pointer supplied by a user program might point into the kernel, into another process's memory, or at an unmapped address.

The kernel must check every pointer and length against the calling process's own address space before dereferencing — and the check must not be defeated by integer overflow in `buf + n`.

It must also **copy** the data into kernel memory rather than operating on it in place, because another thread in the same process could modify it between the validation and the use. That is a **time-of-check to time-of-use** race, and it is a standard exploitation technique rather than a theoretical concern.

A substantial fraction of all kernel vulnerabilities are a missing, incomplete, or racily-defeated check at this step.

3 Three ways into the kernel

Cause	Origin	Synchronou s?	Examples
System call	The program asks, deliberately.	Yes	read, write, fork, exit.
Exception / fault	The program did something the hardware could not complete.	Yes	Page fault, divide by zero, illegal instruction, protection violation.
Interrupt	A device, independently of what is running.	No	Timer tick, disk completion, keyboard, network packet arrival.

The synchronous/asynchronous distinction has real consequences. A system call or fault happens at a known instruction in a known context. An interrupt can arrive *anywhere* — including in the middle of a kernel routine that is halfway through updating a data structure. This is why kernels disable interrupts around critical sections, and why interrupt handlers must be short and must not block.

Note that a **page fault** is an exception, not an error. It is the normal mechanism by which demand paging, copy-on-write, and memory-mapped files work (Module 07). Most page faults are expected and are handled by allocating a page and resuming.

3.1 The timer interrupt

THE INTERRUPT THAT MAKES EVERYTHING ELSE POSSIBLE

Suppose a process enters an infinite loop that makes no system calls. If the kernel only regains control when a program asks for something, it never regains control. The machine hangs.

That was **cooperative** multitasking, and it is exactly why early Windows and Mac systems could be frozen by a single misbehaving application.

The **timer interrupt** fires on a hardware clock regardless of what the running program does. The kernel regains control unconditionally, every few milliseconds, and can choose to run something else.

Preemptive multitasking is precisely this capability. Every scheduling policy in Module 03 presupposes it; without a timer, a scheduler is only a suggestion.

4 Kernel structure

Design	What runs in kernel mode	Trade-off	Examples
Monolithic	Scheduling, memory, file systems, device drivers, network stack — all of it.	Fast: a file system call is a function call. Fragile: a driver bug can corrupt anything.	Linux, Windows NT, xv6, FreeBSD.

Design	What runs in kernel mode	Trade-off	Examples
Microkernel	Only IPC, scheduling, and basic memory management. Drivers and file systems are ordinary user processes.	Robust: a driver crash kills one process. Costly: every service interaction is IPC rather than a call.	seL4, QNX, Minix 3, L4.
Hybrid	Mostly monolithic, with selected services moved out.	A pragmatic middle.	macOS (XNU), Windows in part.

Module 13 returns to this argument with the evidence from the intervening twelve modules. For now it is enough to know that xv6 is monolithic, which is why the whole kernel is about nine thousand lines and can be read in a weekend — which is the point of using it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.1810 — Lecture 1 (Introduction and Examples) and the xv6 book, Chapter 1	Sets up the whole course. Do the setup lab in the same sitting.
OSTEP — Chapters 2 and 4 (Introduction, The Abstraction: The Process)	The abstraction/arbitration framing, written unusually well.
xv6 book — Chapter 4 (Traps and system calls)	The system call path through actual code. Read it with trampoline.S open beside it.
RISC-V Privileged Architecture specification	The hardware contract for privilege modes and traps. Short, clear, and far more readable than the x86 equivalent.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Set up the xv6 toolchain and QEMU. Boot xv6, run `ls`, and attach GDB to the running kernel. Set a breakpoint in `syscall()` and observe a call arriving.
- 2 Trace a single `write()` from user code to the kernel and back, using GDB. Record every mode transition and where the registers are saved.
- 3 Add a system call `getpinfo()` returning the calling process's PID, state, and accumulated run time. This requires touching the syscall table, the user library, and the kernel.
- 4 Write a user program that passes a deliberately invalid pointer to a system call. Confirm the kernel rejects it rather than crashing, and find the validation code that caught it.

- 5 Remove the timer interrupt handler from xv6 and run a program with an infinite loop. Observe the hang, then restore it. This makes preemption concrete.
- 6 Count the lines of code in xv6 by subsystem. Compare against the line count of the Linux kernel and consider what the difference consists of.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the two jobs of an operating system, with two examples of each.
- 2 Name three illusions a kernel maintains and say where each leaks.
- 3 Why can isolation not be implemented in software alone?
- 4 What is a trap vector, and why does it matter that user code cannot choose the kernel entry address?
- 5 Why must the kernel copy data from a user pointer rather than using it in place?
- 6 Name the three ways control enters the kernel and say which is asynchronous.
- 7 Why is the timer interrupt the precondition for preemptive multitasking?